# Guide for GIT

## 📌 1. What is Git?

**Git** is a **Distributed Version Control System (DVCS)** used to:

- Track changes in code
- Collaborate with team
- Maintain version history
- Manage branches & releases

👉 Created by **Linus Torvalds**

---

## 📌 2. Git vs GitHub

| Feature | Git | GitHub |
|---|---|---|
| Type | Tool (Software) | Platform (Cloud) |
| Usage | Version control | Code hosting |
| Works Offline | ☑ Yes | ❌ No |
| Collaboration | Limited | Strong |

👉 GitHub = Online platform built on Git

---

## 📌 3. Git Architecture (Important)

Git has **3 main areas**:

```
Working Directory  →  Staging Area  →  Local Repository
```

| Area | Description |
|---|---|
| Working Directory | Your actual files |
| Staging Area | Ready to commit files |
| Repository | Saved history |

---

## 📌 4. Git Installation & Setup

```
git --version
```

**First time setup:**

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Check config:

```
git config --list
```

---

### 📌 5. Starting a Repository

#### 👉 Initialize new repo

```
git init
```

#### 👉 Clone existing repo

```
git clone <repo_url>
```

---

### 📌 6. Basic Git Workflow

```
1. Modify files
2. Add to staging
3. Commit
4. Push to remote
```

**Commands:**

```
git status
git add filename
git add .
git commit -m "message"
```

---

### 📌 7. Viewing History

```
git log
git log --oneline
git show <commit_id>
```

---

### 📌 8. Branching (🔥 Very Important)

Branch = parallel version of code

**Create branch**

```
git branch feature1
```

**Switch branch**

```
git checkout feature1
```

OR

```
git checkout -b feature1
```

**List branches**

```
git branch
```

---

### 📌 9. Merging Branches

```
git checkout main
git merge feature1
```

**Merge types:**

| Type | Description |
|---|---|
| Fast Forward | Linear merge |
| 3-way merge | With commit |

---

### 📌 10. Merge Conflicts ⚠️

Occurs when:
👉 Same file edited in different branches

Solution:

1. Open file
2. Resolve manually
3. Then:

```
git add .
git commit
```

---

### 📌 11. Undo Changes (VERY IMPORTANT 🔥)

**Unstage file**

```
git reset filename
```

**Undo changes in file**

```
git checkout -- filename
```

**Undo last commit**

```
git reset --soft HEAD~1
```

---

### 📌 12. Stashing (Temporary Save)

```
git stash
git stash pop
git stash list
```

---

### 📌 13. Remote Repository (GitHub)

**Add remote**

```
git remote add origin <url>
```

**Push code**

```
git push -u origin main
```

**Pull code**

```
git pull origin main
```

---

### 📌 14. Fetch vs Pull

| Command | Meaning |
|---|---|
| git fetch | Download changes only |
| git pull | Download + merge |

---

### 📌 15. Rebase (Advanced 🔥)

Rebase = move commits to new base

```
git rebase main
```

👉 Keeps history clean (linear)

---

### 📌 16. Cherry Pick

Pick specific commit

```
git cherry-pick <commit_id>
```

---

### 📌 17. Git Tags (Releases)

```
git tag v1.0
git push origin v1.0
```

---

### 📌 18. Git Ignore

Create `.gitignore` file

Example:

```
node_modules/
*.log
.env
```

---

### 📌 19. Git Diff (Compare changes)

```
git diff
git diff branch1 branch2
```

---

### 📌 20. Git Reflog (Recovery Tool 🚑)

```
git reflog
```

👉 Helps recover deleted commits